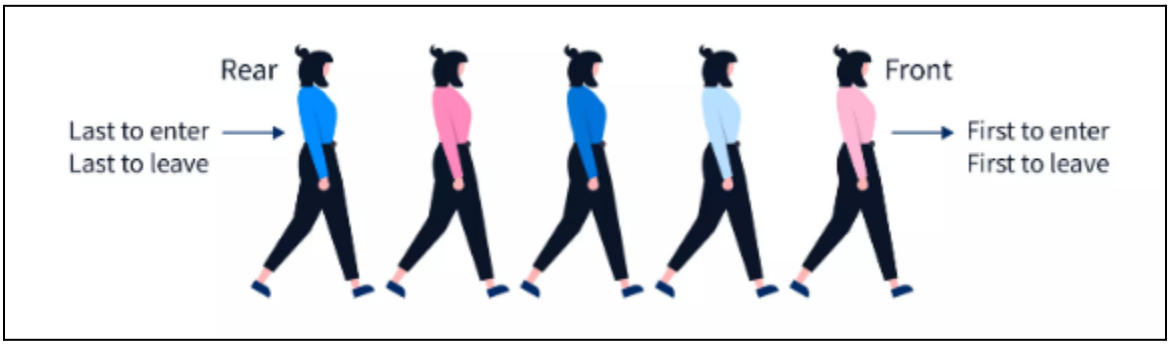
Queue

Similar to other data structures, a queue is a data structure used for storing data. In queue, the order in which data arrives is important. In general, a queue is a line of people or things waiting to be served in sequential order, starting at the beginning of the line or sequence.

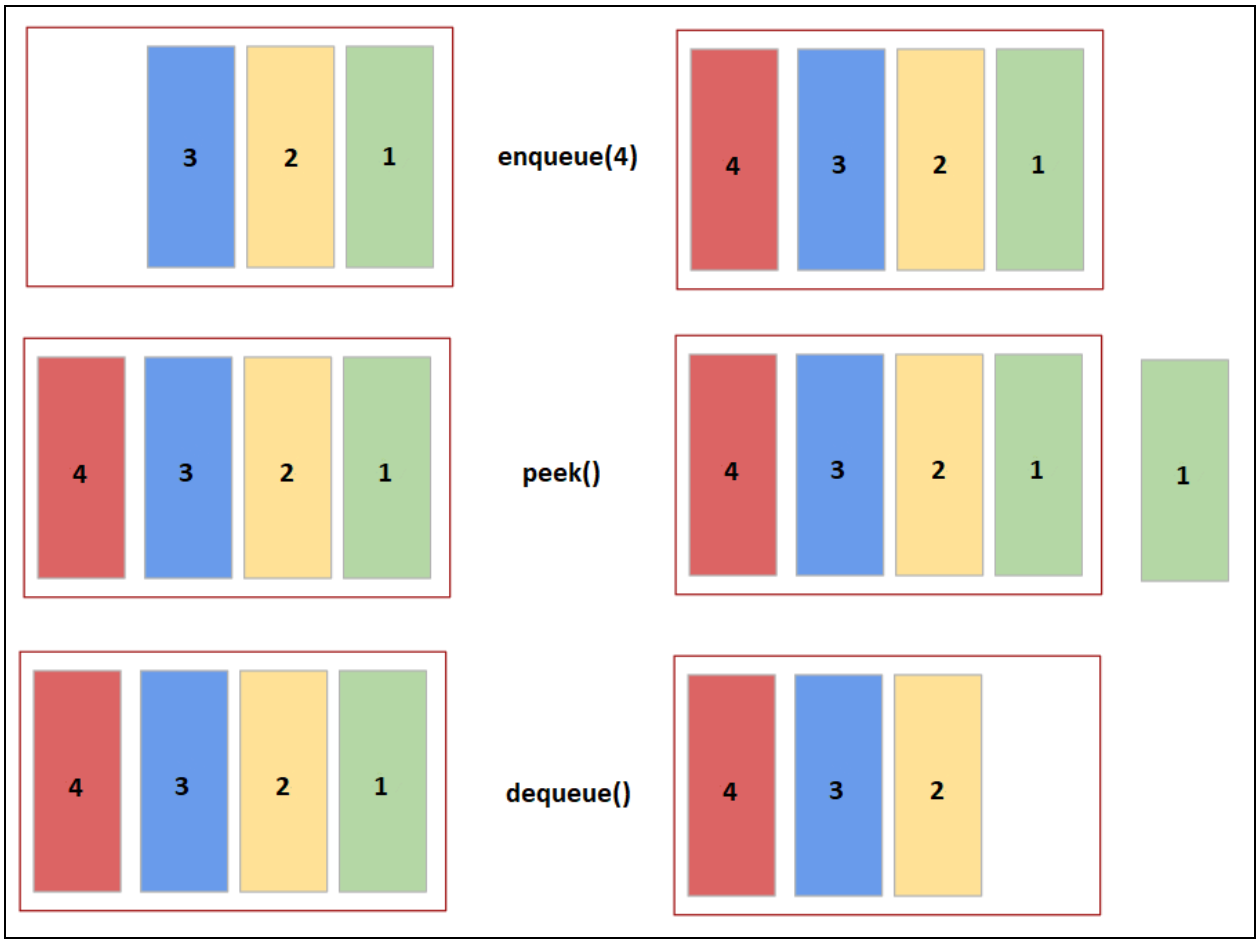


Definition: A queue is an ordered list in which insertions are done at one end (back) and deletions are done at the other end (front). The first element to be inserted is the first one to be deleted. Hence, it is called First in First out (FIFO).

Hence, it's not possible to add or remove elements from the middle of the queue. The only element that can be taken out is the one that came first.

Queue Operations:

- enqueue(obj): It stores a new object in the queue at the end. If the queue is full, this operation throws a QueueOverflow exception.
- dequeue: It removes the first object from the queue. If the queue is empty, this operation throws a QueueUnderflow exception.
- peek: This operation shows the first item in the queue. This operation does not remove any items from the queue.



Queue Implementation:

- Linked List Based Queue

```
class Node:
        function constructor(elem, next):
                self.elem = elem
                self.next = next
```

```
class Queue:
        function constructor():
                self.front = null
                self.rear = null
```

```
function enqueue(elem):
        n = new Node(elem, null)
        if rear == null:
                front = n
                back = n
        else:
                rear.next = n
                rear = n
```

```
function dequeue():
        if front == null:
                print("Queue Underflow")
        else:
                dequeued_item = front.elem
                temp = front
                front = front.next
                temp.next = null
                return dequeued_item
```

```
function peek():
        if front == null:
                print("Queue is empty")
        else:
                return front.elem
```

```
function isEmpty():
        if front == null:
                return True
        else:
                return False
```